

Apply Filters to SQL Queries

Portfolio Activity — SQL Filtering for Security

Investigations Prepared by: G S Krishna Chandra | Google Cybersecurity Certificate

— Course 4 Tools of the Trade: Linux and SQL

Project Description

As a security professional at a large organization, part of my role involves investigating potential security issues using SQL queries against organizational databases. In this project, I used the `log_in_attempts` and `employees` tables to investigate suspicious login activity and identify employee machines for targeted security updates. I applied SQL filters using the `WHERE` clause along with `AND`, `OR`, `NOT`, and `LIKE` operators to retrieve precise subsets of data relevant to each investigation.

Retrieve After Hours Failed Login Attempts

A potential security incident was discovered after business hours. To investigate, I queried the `log_in_attempts` table to find all failed login attempts that occurred after 18:00.

```
select *
-> from log_in_attempts
-> where login_time > '18:00' and success = FALSE;
```

event_id	username	login_date	login_time	country	ip_address	success
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
20	tshah	2022-05-12	18:56:36	MEXICO	192.168.109.50	0
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
52	cjackson	2022-05-10	22:07:07	CAN	192.168.58.57	0
69	wjaffrey	2022-05-11	19:55:15	USA	192.168.100.17	0
82	abernard	2022-05-12	23:38:46	MEX	192.168.234.49	0
87	apatel	2022-05-08	22:38:31	CANADA	192.168.132.153	0
96	ivelasco	2022-05-09	22:36:36	CAN	192.168.84.194	0
104	asundara	2022-05-11	18:38:07	US	192.168.96.200	0
107	bisles	2022-05-12	20:25:57	USA	192.168.116.187	0
111	aestrada	2022-05-10	22:00:26	MEXICO	192.168.76.27	0
127	abellmas	2022-05-09	21:20:51	CANADA	192.168.70.122	0
131	bisles	2022-05-09	20:03:55	US	192.168.113.171	0
155	cgriffin	2022-05-12	22:18:42	USA	192.168.236.176	0
160	jclark	2022-05-10	20:49:00	CANADA	192.168.214.49	0
199	yappiah	2022-05-11	19:34:48	MEXICO	192.168.44.232	0

19 rows in set (0.020 sec)

This query selects all columns from `log_in_attempts` where the `login_time` is later than 18:00 **AND** the `success` column equals 0 (a failed attempt). The `AND` operator ensures both conditions must be true for a row to be returned — narrowing the results to only failed logins that occurred after hours.

Retrieve Login Attempts on Specific Dates

A suspicious event occurred on 2022-05-09. To investigate, I queried for all login attempts on that date and the day before, 2022-05-08.

```
MariaDB [organization]> select *  
  -> from log_in_attempts  
  -> where login_date = '2022-05-09' or login_date = '2022-05-08';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
15	lyamamot	2022-05-09	17:17:26	USA	192.168.183.51	0
24	arusso	2022-05-09	06:49:39	MEXICO	192.168.171.192	1
25	sbaelish	2022-05-09	07:04:02	US	192.168.33.137	1
26	apatel	2022-05-08	17:27:00	CANADA	192.168.123.105	1
28	aestrada	2022-05-09	19:28:12	MEXICO	192.168.27.57	0
30	yappiah	2022-05-09	03:22:22	MEX	192.168.124.48	1
32	acook	2022-05-09	02:52:02	CANADA	192.168.142.239	0
36	asundara	2022-05-08	09:00:42	US	192.168.78.151	1
38	sbaelish	2022-05-09	14:40:01	USA	192.168.60.42	1
39	yappiah	2022-05-09	07:56:40	MEXICO	192.168.57.115	1
42	cgriffin	2022-05-09	23:04:05	US	192.168.4.157	0
43	mcouliba	2022-05-08	02:35:34	CANADA	192.168.16.208	0

This query returns all rows where login_date equals '2022-05-09' **OR** equals '2022-05-08'. The OR operator returns a row if at least one of the conditions is true, allowing the query to capture login attempts across both dates in a single result set.

Retrieve Login Attempts Outside of Mexico

The team determined that suspicious activity didn't originate in Mexico, so I queried for all login attempts that occurred outside of Mexico. The country column contains values of both MEX and MEXICO, so I used LIKE with a wildcard to match both.

```
MariaDB [organization]> select *
-> from log_in_attempts
-> where NOT country like 'MEX%';
```

event_id	username	login_date	login_time	country	ip_address	success
1	jrafael	2022-05-09	04:56:27	CAN	192.168.243.140	1
2	apatel	2022-05-10	20:27:27	CAN	192.168.205.12	0
3	dkot	2022-05-09	06:47:41	USA	192.168.151.162	1
4	dkot	2022-05-08	02:00:39	USA	192.168.178.71	0
5	jrafael	2022-05-11	03:05:59	CANADA	192.168.86.232	0
7	eraab	2022-05-11	01:45:14	CAN	192.168.170.243	1
8	bisles	2022-05-08	01:30:17	US	192.168.119.173	0
10	jrafael	2022-05-12	09:33:19	CANADA	192.168.228.221	0
11	sgilmore	2022-05-11	10:16:29	CANADA	192.168.140.81	0
12	dkot	2022-05-08	09:11:34	USA	192.168.100.158	1
13	mrhah	2022-05-11	09:29:34	USA	192.168.246.135	1
14	sbaelish	2022-05-10	10:20:18	US	192.168.16.99	1
15	lyamamot	2022-05-09	17:17:26	USA	192.168.183.51	0
16	mcouliba	2022-05-11	06:44:22	CAN	192.168.172.189	1
17	pwashing	2022-05-11	02:33:02	USA	192.168.81.89	1
18	pwashing	2022-05-11	19:28:50	US	192.168.66.142	0
19	jhill	2022-05-12	13:09:04	US	192.168.142.245	1
21	iuduik	2022-05-11	17:50:00	US	192.168.131.147	1
25	sbaelish	2022-05-09	07:04:02	US	192.168.33.137	1
26	apatel	2022-05-08	17:27:00	CANADA	192.168.123.105	1
29	bisles	2022-05-11	01:21:22	US	192.168.85.186	0
31	acook	2022-05-12	17:36:45	CANADA	192.168.58.232	0
32	acook	2022-05-09	02:52:02	CANADA	192.168.142.239	0
33	zbernal	2022-05-11	02:52:10	US	192.168.72.59	1
34	drosas	2022-05-11	21:02:04	US	192.168.45.93	0
36	asundara	2022-05-08	09:00:42	US	192.168.78.151	1
37	eraab	2022-05-10	06:03:41	CANADA	192.168.152.148	0
38	sbaelish	2022-05-09	14:40:01	USA	192.168.60.42	1

The LIKE keyword matches a pattern rather than an exact value. The % wildcard matches any sequence of characters after 'MEX', so 'MEX%' matches both MEX and MEXICO. The NOT operator reverses the condition, so this query returns only rows where the country does **not** match either of those values — i.e., all login attempts that occurred outside Mexico.

Retrieve Employees in Marketing

The team needed information on Marketing department machines located in the East building, in order to perform security updates. I queried the employees table for all employees in Marketing whose office is in the East building (e.g., East-170, East-320).

```
MariaDB [organization]> select * from employees where department = 'Marketing' and office like 'East%';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1052	a192b174c940	jdarosa	Marketing	East-195
1075	x573y883z772	fbautist	Marketing	East-267
1088	k865l965m233	rgosh	Marketing	East-157
1103	NULL	randerss	Marketing	East-460
1156	a184b775c707	dellery	Marketing	East-417
1163	h679i515j339	cwilliam	Marketing	East-216

```
7 rows in set (0.001 sec)
```

This query returns rows where department equals 'Marketing' **AND** office starts with 'East' (using the % wildcard to match any office number, e.g. East-170 or East-320). Both conditions must be true, so only Marketing employees located specifically in East building offices are returned.

Retrieve Employees in Finance or Sales

A separate security update needed to be applied to machines for employees in the Sales and Finance departments. I queried the employees table for employees in either department.

```
MariaDB [organization]> select * from employees where department = 'Finance' or department = 'Sales';
```

employee_id	device_id	username	department	office
1003	d394e816f943	sgilmore	Finance	South-153
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodriqu	Sales	South-134
1010	k242l212m542	jlansky	Finance	South-109
1011	l748m120n401	drosas	Sales	South-292
1015	p611q262r945	jsoto	Finance	North-271
1017	r550s824t230	jclark	Finance	North-188
1018	s310t540u653	abellmas	Finance	North-403

This query returns rows where department equals 'Sales' **OR** equals 'Finance'. Since OR requires only one condition to be true, this returns all employees from either department in a single result set, useful for applying the same update to both groups at once.

Retrieve All Employees Not in IT

Employees in the Information Technology department already received a particular security update, so the remaining update needed to target everyone else. I queried the employees table for all employees not in the IT department.

```
MariaDB [organization]> select * from employees where not department = 'Information Technology';
```

employee_id	device_id	username	department	office
1000	a320b137c219	elarson	Marketing	East-170
1001	b239c825d303	bmoreno	Marketing	Central-276
1002	c116d593e558	tshah	Human Resources	North-434
1003	d394e816f943	sgilmore	Finance	South-153
1004	e218f877g788	eraab	Human Resources	South-127
1005	f551g340h864	gesparza	Human Resources	South-366
1007	h174i497j413	wjaffrey	Finance	North-406
1008	i858j583k571	abernard	Finance	South-170
1009	NULL	lrodrigu	Sales	South-134

This query uses the `!=` (not equal to) operator, equivalent to `NOT department = 'Information Technology'`, to return all rows where the department column does not match 'Information Technology'. This excludes IT employees from the result, ensuring the update is applied only to employees in all other departments.

Summary

In this project, I used SQL filters to investigate security incidents and support targeted security updates across an organization's systems. I applied the AND operator to isolate after-hours failed login attempts, the OR operator to retrieve login activity across multiple dates and departments, the LIKE operator with wildcards to match patterns in country and office values, and the NOT operator to exclude specific departments from results. Together, these queries demonstrate how SQL filtering can be used to efficiently narrow large datasets down to the exact records needed for a security investigation or operational task.